`djago-admin`. To create a new Django project, you use the `startproject` command, giving it the name of your project. By default, `startproject` creates a new directory for your project, copies a script called manage.py inside, and creates a configuration directory. The manage.py script is similar to `django-admin` and is used to run various control and configuration actions in the project. The configuration directory contains wsgi.py, urls.py, and other files.

The configuration directory is confusingly named: it has the same name as your project. When you create RiffMates, you're going to get both a RiffMates directory and a RiffMates/RiffMates directory. It is unclear to me why the Django folks didn't name this something like *config*, as this causes confusion when you're talking to someone else about the code: Are you in the RiffMates directory? "Yes." Which one?

Naming your project

Python modules are typically named using *snake case*, or lowercase words separated by underscores. I'm not sure where I picked up the habit of naming my Django projects in violation of this convention—it may be because I'm an ex-Java programmer. If seeing *RiffMates* makes you cringe, feel free to name your project in a more "Pythonic" fashion.

Because your Django project is a directory itself, you have some flexibility about where you put your Python virtual environment. Some virtual environment tools prefer keeping all the environments together in a common directory, whereas others keep the environment with the project. In this latter case, you can put your virtual environment inside your project folder, which is what I'll be doing in the example here.

To create your first Django project, you're going to do the following:

1. Create a virtual environment.
2. Install Django into that virtual environment.
3. Use the Django `django-admin` tool to create your RiffMates project.

Once you've done all of this, you will be able to start the development server and point your browser at your new site. Open a command-line terminal, and change to the directory where you normally keep your code. Inside the terminal, create a directory for your project and a virtual environment inside of it:

```
$ mkdir RiffMates
$ cd RiffMates
RiffMates$ python3.12 -m venv ./venv
```

Create a directory for your project.

Create a virtual environment.

NOTE Most of this book will be spent in Python, but to get going, you need to run some command-line scripts. All command-line examples use Unix syntax with dollar-sign ($) denoting the prompt. The value before the dollar-sign indicates the current directory and whether a virtual environment has been activated. For details on how to install Unix-compatible tools on Windows, see appendix A.